

Extended Essay – Computer Science

Topic:

K-Means Applied to Handwritten Digit Recognition

Research Question:

To what extent can the performance of the K-Means clustering algorithm be optimized on the MNIST dataset using different parameter settings in scikit-learn?

Subject: Computer Science

Word Count: 3998

Contents

1	Introduction	2
2	Background	4
2.1	Supervised and Unsupervised Learning	4
2.1.1	Clustering Algorithms	4
2.1.2	K-Means Clustering	5
2.1.3	Commonly used Evaluation Metrics in Clustering	7
2.2	The MNIST Dataset	7
2.2.1	Justification for Dataset Selection	8
3	Methodology	9
3.1	Experimental Setup	9
3.1.1	Device Information	9
3.1.2	Execution Environment	9
3.1.3	Preprocessing	9
3.2	Design Overview	9
3.2.1	Experiment 1: Broad Parameter Exploration	10
3.2.2	Experiment 2: Targeted Parameter Refinement	10
3.2.3	Evaluation Metrics	11
4	Results and Analysis	12
4.1	Analysis of Experiment 1 Results	12
4.1.1	Transition from Experiment 1 to Experiment 2	16
4.2	Experiment 2 Results – Targeted Refinement	17
5	Conclusion	23
6	Evaluation	25
6.1	Evaluation of the Investigation	25
6.2	Strengths and Limitations	25
7	Works Cited	27
8	Appendices	29
A.	Code for Experiment 1	29
B.	Code for Experiment 2	33
C.	Code for generating accuracy	36

1 Introduction

As the world continues its technological advancements and growth, the field of machine learning has, in recent years, developed at an extraordinary pace, revolutionizing the way humans interact with the world around them. From seemingly ordinary tasks such as recommending contents on streaming platforms, personalizing search results and filtering spam emails, to much more impactful innovations including medical diagnostics, transportation planning and financial fraud detection, machine learning is deeply embedded into our everyday lives. The ability of machine learning to learn from data, identify patterns and make “intelligent decisions” has not only enhanced convenience but is also reshaping entire industries and societal structures.

One area where the impact of machine learning is especially evident is in digital security. CAPTCHA systems, short for “Completely Automated Public Turing test to tell Computers and Humans Apart”, are designed to exploit the difficulty machines have in recognizing distorted text or images, hence aiming to identify machines from humans (Hidalgo and Alvarez). While pattern recognition remains intuitive for humans, they have long posed challenges for machines. There are different types of CAPTCHAs (Figure 1), among these, handwritten digit and letter recognition is one of the most long standing and widely used method.



Figure 1: Commonly seen CAPTCHAs (cite - Noury)

This study therefore focuses on handwritten digit recognition, using the MNIST dataset as a benchmark and the K-Means clustering algorithm as the method of investigation. Specifically, this essay explores how changes in number of clusters, number of initializations, and maximum iterations affect the quality of clustering and runtime. The goal is to evaluate whether K-Means is an effective unsupervised learning tool for complex image data and under what configurations it performs best.

Hence, the central research question guiding this investigation is:

To what extent can the performance of the K-Means clustering algorithm be optimized on the MNIST dataset using different parameter settings in scikit-learn?

2 Background

Machine learning is a branch of artificial intelligence that enables computers to learn from data and make predictions or identify patterns without being explicitly programmed. (Mitchell 2) It has become a fundamental tool in modern computing, powering applications in image recognition, natural language processing, recommendation systems, and fraud detection. At its core, machine learning allows systems to improve their performance over time through experience, rather than relying solely on pre-defined rules.

2.1 Supervised and Unsupervised Learning

Machine learning methods are typically divided into two main categories: **supervised learning** and **unsupervised learning** (Jiang, 2020).

In supervised learning, algorithms are trained on labeled datasets, where the input data is paired with the correct output. This allows the model to learn a mapping function and make predictions on new, unseen data. (Jiang, 2020) Common tasks include:

- **Classification:** e.g., recognizing whether an email is spam or not.
- **Regression:** e.g., predicting house prices.

In contrast, unsupervised learning deals with unlabeled data—where the algorithm must identify patterns or groupings on its own without guidance (Jain, 2010). One of the most widely used unsupervised learning techniques is **clustering**, which involves grouping similar data points together based on their characteristics. Clustering is especially useful when dealing with high-dimensional data such as images, where clear patterns may not be immediately obvious. Hence, clustering is chosen to be explored in this investigation.

2.1.1 Clustering Algorithms

Over decades of research, many clustering algorithms have been developed, each with different assumptions and strengths (Xu & Wunsch, 2005). In general, clustering methods fall into a few broad categories:

- **Hierarchical Clustering:** These algorithms build a hierarchy of clusters either by *agglomeration* (starting with each point as its own cluster and merging progressively) or *division* (starting with one cluster and splitting it) (Murtagh & Contreras, 2012). The output is often a dendrogram (tree) that can be cut at a desired level to yield a flat clustering (Wani, 3-4).
- **Density-Based Clustering (DBSCAN and variants):** Density-based methods such as DBSCAN (Density Based Spatial Clustering of Applications with Noise)

identify clusters as regions of high point density separated by regions of low density (Ester et al., 1996).

- **Partition-Based Clustering (K-Means and variants):** These methods directly partition the dataset into a set number of clusters. The classic example is K-Means, introduced by MacQueen in 1967 (MacQueen, 1967). K-Means iteratively assigns points to the nearest of k centroids and updates those centroids to minimize within-cluster variance. It requires the user to specify the number of clusters k in advance and typically uses Euclidean distance.
- **Other methods:** Less commonly used approaches include model-based clustering and spectral clustering (Wani 9-11).

Given this diversity, it is well understood that no single clustering algorithm works best for all problems—each imposes a certain structure or bias, and the “right” choice depends on the dataset characteristics. Indeed, experts observed that “there is no single clustering algorithm that has been shown to dominate other algorithms across all application domains,” so algorithm selection should be guided by the data and goals (Jain, 2010).

Among these methods, **K-Means** is one of the most widely applied due to its conceptual simplicity and computational efficiency. It partitions the dataset into a predefined number of clusters by iteratively refining centroid positions. Because of these properties, K-Means is selected for detailed investigation in this essay and will be discussed in depth in the following subsection.

2.1.2 K-Means Clustering

K-Means is one of the most widely used partition-based clustering methods due to its simplicity, efficiency, and interpretability (Jain, 2010). It aims to divide a dataset into k clusters, where each data point belongs to the cluster with the nearest centroid, thereby minimizing the within-cluster variance, also known as inertia (“2.3. Clustering”). The algorithm iteratively assigns points to the closest centroid and updates centroid positions until convergence. Because of its speed and conceptual clarity, K-Means is often chosen as a baseline method in clustering research.

In practice, however, K-Means performs best under a set of assumptions about the underlying data distribution. First, it implicitly assumes that clusters are roughly spherical in shape, meaning that data points belonging to the same cluster are distributed around a centroid in all directions with comparable spread. Second, it assumes that clusters have similar variance (isotropy), so that no cluster is disproportionately elongated or dense compared to others. Finally, K-Means relies on the Euclidean distance metric as its measure of similarity, which presupposes that distances between points can be meaningfully captured in this linear geometric space.

For the K-means algorithm, we assume an integer k and a set of n data points $X \subset \mathbb{R}^d$. The K-means problem is to position a set

$$C = \{c_1, c_2, \dots, c_k\} \quad (1)$$

of k d -dimensional centroids such that the error function

$$\phi(C, X) = \sum_{x \in X} \min_{c \in C} \|x - c\|^2 \quad (2)$$

is minimized (Fritzke). We will also refer to $\phi(C, X)$ as the **Summed Squared Error (SSE)**. In the context of vector quantization, the centroid set C is called a *codebook*, centroids are referred to as *codebook vectors*, and $\phi(C, X)$ is denoted as *quantization error* (Fritzke).

Algorithm 1 k -means algorithm ((Arvai))

- 1: Specify the number k of clusters to assign.
 - 2: Randomly initialize k centroids.
 - 3: **repeat**
 - 4: **Expectation:** Assign each point to its closest centroid.
 - 5: **Maximization:** Compute the new centroid (mean) of each cluster.
 - 6: **until** The centroid positions do not change.
-

While conceptually straightforward, the performance of K-Means can vary significantly depending on the choice of parameters. According to the scikit-learn documentation, key parameters include:

- **n_clusters:** the number of clusters (k), which must be defined in advance.
- **n_init:** the number of times the algorithm is run with different centroid seeds to avoid local minima.
- **max_iter:** the maximum number of iterations for each run.
- **init:** the method used to initialize the centroids. In this investigation, it is fixed to **k-means++**, which is also the default setting, as it spreads initial centroids far apart and hence improves convergence.
- **random_state:** the seed value for reproducibility. Specific seeds were chosen and recorded in case a clustering attempt needed to be reproduced.
- **algorithm:** the computational method used for updating clusters. The commonly used option is **lloyd** (Lloyd’s algorithm). However, after investigation it was decided that **elkan**, an optimized variant of Lloyd’s algorithm, is faster at convergence with Euclidean distance (“KMeans”).

2.1.3 Commonly used Evaluation Metrics in Clustering

Evaluating the quality of clustering is not straightforward, since the task is inherently unsupervised and ground truth labels are usually unavailable. To address this, researchers typically divide evaluation metrics into two categories:

- **Internal metrics**, which do not require labels and instead measure the clustering structure itself. These metrics assess how compact and well-separated clusters are. A common example is the *Silhouette Score*, which combines intra-cluster cohesion with inter-cluster separation (Rousseeuw, 1987). Other internal measures include the Davies–Bouldin index and Calinski–Harabasz index, which also capture cluster compactness and distinctness (Xu, 2005).
- **External metrics**, which rely on comparing clustering results against ground truth labels (when available). These include the Adjusted Rand Index (ARI), Adjusted Mutual Information (AMI), and Accuracy (Jain, 2010). External metrics are particularly valuable when datasets like MNIST provide true class labels, as they allow clustering performance to be interpreted in terms of classification-like outcomes.

However, since this experiment is a very specific one, and the labels of each digit are available, an own designed Evaluation Method will be employed.

2.2 The MNIST Dataset

The MNIST (Modified National Institute of Standards and Technology) dataset is one of the most established and frequently cited datasets in the field of machine learning (LeCun et al. 1998). It consists of 70,000 grayscale images of handwritten digits from 0 to 9, each sized at 28×28 pixels (LeCun et al. 1998). The dataset was constructed by Yann LeCun and collaborators by combining and preprocessing samples from the larger NIST Special Database 1 and Special Database 3, originally collected from U.S. Census Bureau employees and high school students (LeCun, “The MNIST Database”).

MNIST’s popularity stems from its well-balanced class distribution, consistent image size, and absence of significant noise or distortions. Each image is centered and size-normalized, allowing researchers to focus on algorithmic performance without the added complexity of extensive data cleaning or preprocessing (LeCun et al. 1998). Because of these qualities, MNIST has become a *de facto* starting point for evaluating new classification and clustering methods (Deng 2012).

2.2.1 Justification for Dataset Selection

The decision to use MNIST is grounded in its status as a clean, low-variability dataset that provides a reliable baseline for evaluating algorithm performance. It offers a clear starting point for parameter tuning and visualization, allowing for controlled experimentation without the confounding effects of noise or significant class overlap.

By focusing on MNIST, this essay can concentrate on analyzing the relationship between parameter choices and clustering performance, ensuring that observed results are attributable to the algorithmic design rather than unpredictable dataset complexity.

3 Methodology

3.1 Experimental Setup

3.1.1 Device Information

- Device Name: MacBook Pro
- Model Identifier: Mac15,3
- Model Number: MTL73ZP/A
- Chip: Apple M3
- Total Number of Cores: 8 (4 performance and 4 efficiency)
- Memory: 8 GB
- System Firmware Version: 11881.81.2

3.1.2 Execution Environment

- Python 3.11
- IDE: Visual Studio Code 1.101.1
- Libraries: scikit-learn, numpy, matplotlib

3.1.3 Preprocessing

The MNIST dataset underwent minimal preprocessing to ensure no loss of datapoints:

- The dataset was downloaded directly from `scikit-learn`'s `openml`.
- No pixel value normalization or dimensionality reduction was performed.
- The original 28×28 pixel images (flattened into 784-dimensional vectors) were used directly as input to K-Means.

This preserved all raw image information for clustering.

3.2 Design Overview

This investigation was conducted in two experiments to balance broad exploration of parameter effects with targeted refinement based on preliminary findings:

- **Experiment 1 (Broad Parameter Exploration):** A wide range of values for the main K-Means parameters were tested (including number of clusters, number of initializations, and number of iterations), with multiple trials (2 trials) to investigate the effects of random variations. The aim was to identify general trends in accuracy and runtime.
- **Experiment 2 (Targeted Refinement):** Focused on the most relevant parameters and extended the range of the number of clusters based on Experiment 1 observations. Redundant parameter combinations were removed to improve efficiency and concentrate on unexplored or high-impact regions.

This two-experiment structure ensured that initial findings informed the refinement process, avoiding unnecessary computation while still providing a thorough analysis of K-Means behaviour on MNIST.

3.2.1 Experiment 1: Broad Parameter Exploration

In Experiment 1, the goal was to test a broad range of values for `n_clusters`, `n_init`, and `max_iter` to observe general performance patterns.

Table 1: Parameters tested in Experiment 1

Parameter	Description	Values Tested
<code>n_clusters</code>	Number of clusters	10, 25, 50, 75, 100
<code>n_init</code>	Number of centroid initializations	10, 20, 50
<code>max_iter</code>	Maximum iterations per cluster run	100, 300, 500

Procedure: Each configuration was run twice with different random seeds. Accuracy and runtime were recorded for each trial. Results were averaged to minimize the effect of random variation.

3.2.2 Experiment 2: Targeted Parameter Refinement

Experiment 1 results showed that:

- Higher values of `n_init` had negligible effect on clustering quality but increased runtime.
- Performance improvements plateaued beyond `max_iter` = 100.
- The effect of `n_clusters` was more nuanced, with potential trends emerging at higher values not covered in Experiment 1.

Based on these findings, Experiment 2 was designed to focus on:

- Lower `n_init` values (1–3) to reduce redundancy.
- Extended `n_clusters` range (50–350) to explore higher cluster counts.
- Reduced `max_iter` values (50 and 100) to concentrate on efficient configurations.
- Single runs per configuration, as Experiment 1 showed low run-to-run variability.

Table 2: Parameters tested in Experiment 2

Parameter	Description	Values Tested
<code>n_clusters</code>	Number of clusters	50, 100, 150, 200, 250, 300, 350
<code>n_init</code>	Number of centroid initializations	1, 2, 3
<code>max_iter</code>	Maximum iterations per cluster run	50, 100

3.2.3 Evaluation Metrics

After clustering, these criteria are used to assess the performance:

Quantitative evaluation using clustering metrics:

- **Accuracy:** This is an external metric and will be the main evaluative metric in this investigation. I defined Accuracy by addigning each cluster to the digit label most closely within it and then calcuating the percentage of correctly assigned digits across the dataset. This use, although not commonly seen in unsupervised contexts, it is justified here as it enables clear comparison of parameter effects on clustering performance. The final accuracy is presented as a percentage of correctly categorized digits.
- **Computation Time:** This will be the total computational time needed for the clustering process, measured using Python’s `time.time()` function.

Visualization of cluster assignments:

- Reconstructing representative digit images from each cluster.
- Constructing the digit image of each cluster center.

These metrics were selected to balance clustering accuracy with computational efficiency.

4 Results and Analysis

4.1 Analysis of Experiment 1 Results

Experiment 1 involved 45 unique parameter configurations, each tested twice with different random seeds to account for variability. Accuracy and runtime were recorded for every configuration, and the results were averaged across the two trials. The overall trends provide an initial baseline for understanding the effect of the key parameters `n_clusters`, `n_init`, and `max_iter` on K-Means performance with MNIST.

The most consistent finding was the influence of `n_clusters` on accuracy. Performance rose sharply as the number of clusters increased from 10 to 100, with average accuracy improving from approximately 57% at `n_clusters` = 10 to nearly 87% at `n_clusters` = 100. Gains beyond ~ 75 clusters were smaller, indicating a point of diminishing returns within the tested range. This pattern is illustrated in Figure 2, where accuracy increases are most pronounced in the lower range of cluster values before tapering off toward the upper end.

The parameter `max_iter` had a subtler effect. Increasing from 100 to 300 iterations yielded small improvements in some cases, particularly at lower `n_clusters`. Beyond 300, however, changes in accuracy were negligible, confirming that most runs converged well before 100 iterations. This early convergence suggests that larger iteration limits unnecessarily long runtime without significant accuracy gains.

By contrast, `n_init` had minimal impact on accuracy. Increasing from 10 to 50 restarts produced no meaningful improvement in clustering quality but did scale runtime nearly proportionally. For instance, at `n_clusters` = 100 and `max_iter` = 100, runtime increased from roughly 31 seconds at `n_init` = 10 to more than 155 seconds at `n_init` = 50, while accuracy remained steady at around 87%. This highlights the computational inefficiency of large initialization counts.

Runtime patterns confirmed the combined effect of all three parameters. Time requirements rose consistently with larger `n_clusters`, higher `n_init`, and larger `max_iter`. The fastest configurations, such as `n_clusters` = 10 with `n_init` = 10 and `max_iter` = 100, completed in just over 7 seconds. The slowest, with `n_clusters` = 100, `n_init` = 50, and `max_iter` = 500, required close to 3 minutes. Importantly, the trade-off between runtime and performance was not linear: large increases in runtime often produced little or no corresponding gain in accuracy.

Finally, stability across trials was strong. Variation between the two runs of the same configuration was minimal, typically below 0.5% in accuracy and within a few seconds in runtime. This consistency confirms that random seed effects had limited influence on MNIST under K-Means clustering, leading to the decision in Experiment 2 to run each configuration only once.

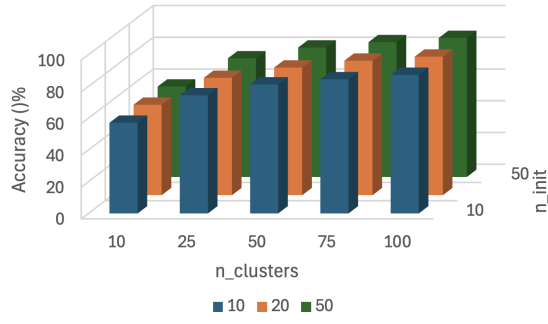
These results establish a basic understanding of parameter effects and highlight clear inefficiencies in the initial search space. They also demonstrate how raw, unscaled pixel data could be clustered effectively without preprocessing, meaning that observed changes in performance were attributable directly to parameter variation rather than data transformation.

Table 3: Experiment 1 Results – Time and Accuracy across Trials

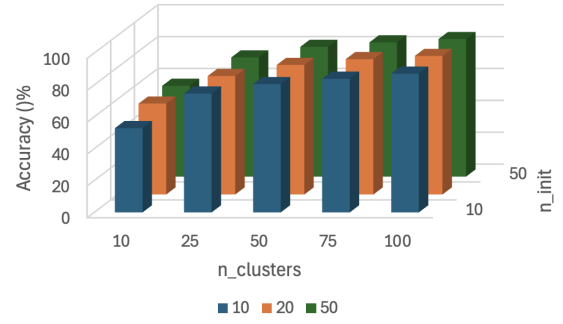
max_iter	n_init	n_clusters	TIME (s)			ACCURACY (%)		
			Trial 1	Trial 2	AVG	Trial 1	Trial 2	AVG
100	10	10	7.2	7.4	7.3	57.1	57.2	57.1
100	10	25	12.4	12.4	12.4	74.1	74.7	74.4
100	10	50	19.5	18.5	19.0	81.5	81.1	81.3
100	10	75	25.3	25.4	25.4	84.5	84.6	84.5
100	10	100	31.4	31.5	31.5	87.2	87.5	87.4
100	20	10	13.9	14.1	14.0	57.0	57.1	57.0
100	20	25	24.6	23.9	24.2	74.0	73.9	74.0
100	20	50	36.2	37.3	36.8	80.6	80.2	80.4
100	20	75	52.1	50.2	51.1	85.0	84.6	84.8
100	20	100	65.6	64.1	64.9	87.8	86.8	87.3
100	50	10	34.7	32.9	33.8	57.0	57.0	57.0
100	50	25	59.9	59.7	59.8	74.7	74.8	74.8
100	50	50	96.4	95.2	95.8	81.6	81.6	81.6
100	50	75	127.8	129.0	128.4	84.8	85.0	84.9
100	50	100	155.1	156.2	155.7	87.4	87.8	87.6
300	10	10	8.4	6.4	7.4	52.9	52.9	52.9
300	10	25	13.8	13.1	13.4	74.9	74.3	74.6
300	10	50	23.2	22.1	22.6	80.0	81.0	80.5
300	10	75	29.0	31.0	30.0	84.1	83.7	83.9
300	10	100	38.0	35.7	36.9	87.0	87.1	87.0
300	20	10	15.3	15.2	15.3	57.0	57.2	57.1
300	20	25	25.7	29.0	27.4	74.8	73.9	74.3
300	20	50	45.2	44.7	45.0	81.3	81.3	81.3
300	20	75	56.7	57.5	57.1	86.1	83.9	85.0
300	20	100	67.8	72.5	70.1	87.5	86.4	87.0
300	50	10	38.3	35.5	36.9	57.0	57.0	57.0
300	50	25	68.1	65.6	66.9	74.8	74.8	74.8
300	50	50	110.8	111.1	111.0	80.9	82.0	81.5
300	50	75	146.5	146.4	146.4	84.0	84.5	84.2

Table 3: Experiment 1 Results – continued

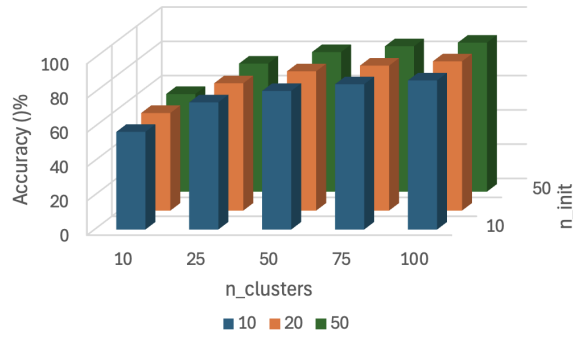
max_iter	n_init	n_clusters	TIME (s)			ACCURACY (%)		
			Trial 1	Trial 2	AVG	Trial 1	Trial 2	AVG
300	50	100	176.0	177.2	176.6	85.9	86.7	86.3
500	10	10	8.7	8.1	8.4	57.0	57.0	57.0
500	10	25	16.8	14.8	15.8	74.2	74.2	74.2
500	10	50	22.3	25.1	23.7	81.4	80.3	80.9
500	10	75	29.6	31.7	30.7	84.8	84.8	84.8
500	10	100	37.7	37.0	37.4	87.0	87.0	87.0
500	20	10	13.6	15.2	14.4	57.0	57.0	57.0
500	20	25	28.9	28.8	28.8	73.9	74.7	74.3
500	20	50	44.9	45.1	45.0	80.9	81.7	81.3
500	20	75	56.3	57.8	57.0	85.0	84.2	84.6
500	20	100	71.6	68.8	70.2	87.4	86.8	87.1
500	50	10	40.3	38.0	39.1	57.0	57.0	57.0
500	50	25	64.9	65.6	65.2	74.8	74.7	74.8
500	50	50	108.4	114.0	111.2	81.7	81.2	81.5
500	50	75	152.9	149.7	151.3	84.7	85.1	84.9
500	50	100	176.2	178.0	177.1	87.1	86.8	87.0



(a) graph of n_cluster and n_init against accuracy, with max_iter of 100

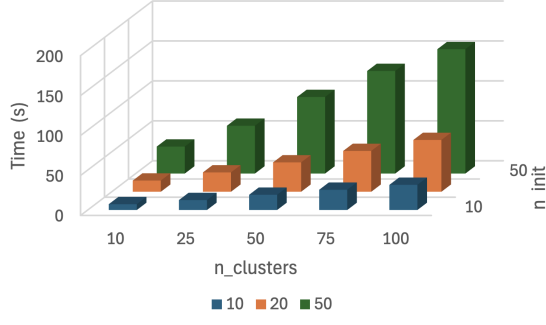


(b) graph of n_cluster and n_init against accuracy, with max_iter of 300

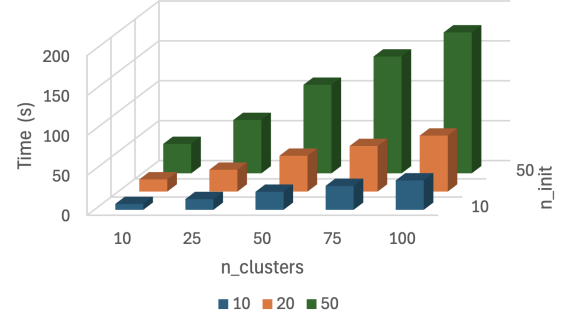


(c) graph of n_cluster and n_init against accuracy, with max_iter of 500

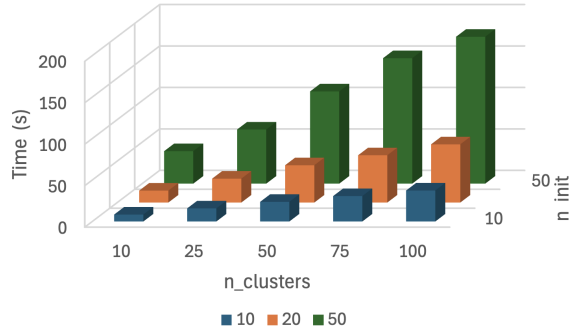
Figure 2: Processed accuracy graphs across parameter settings.



(a) graph of $n_cluster$ and n_init against time, with max_iter of 100



(b) graph of $n_cluster$ and n_init against time, with max_iter of 300



(c) graph of $n_cluster$ and n_init against time, with max_iter of 500

Figure 3: Processed time graphs across parameter settings.

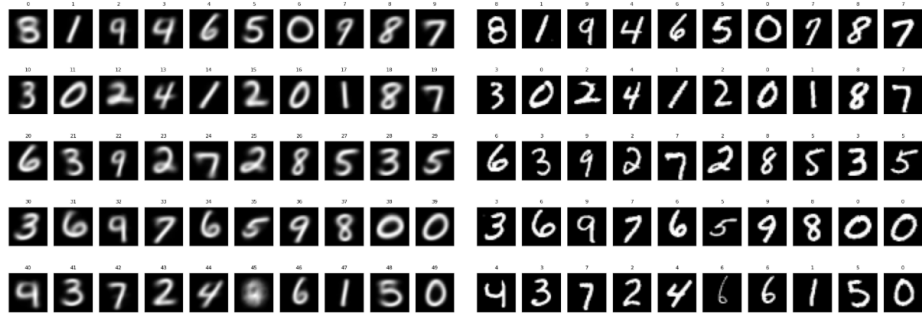


Figure 4: Example of cluster centers (left) and their nearest instances (right) for $n_clusters = 50$, $n_init = 10$, $max_iter = 100$, $seed = 3$.

4.1.1 Transition from Experiment 1 to Experiment 2

The analysis of Experiment 1 show several consistent patterns that helped the design of the second experiment. It was noticed that increasing n_init , the initialization count, beyond a small value produced only negligible improvements in accuracy, while substantially increasing computation time. Accuracy also tended to plateau once max_iter exceeded

100, suggesting that longer runs offered little additional benefit and that convergence was typically reached earlier. Finally, although the tested range for `n_clusters` in Experiment 1 (10–100) showed a clear positive trend toward higher cluster counts, its performance at even larger values remains undetermined.

On the basis of these observations, Experiment 2 refined the parameter space to emphasize efficiency and coverage of unexplored ranges.

4.2 Experiment 2 Results – Targeted Refinement

Experiment 2 tested 42 configurations chosen on the basis of the trends identified in Experiment 1. Specifically, the number of clusters (`n_clusters`) was extended up to 350 in order to evaluate whether accuracy gains continued at higher cluster counts, `n_init` was reduced to values between 1 and 3 to remove redundancy, and `max_iter` was limited to 50 and 100 to concentrate on convergence efficiency. Each configuration was executed once, since Experiment 1 had shown that run-to-run variability was negligible.

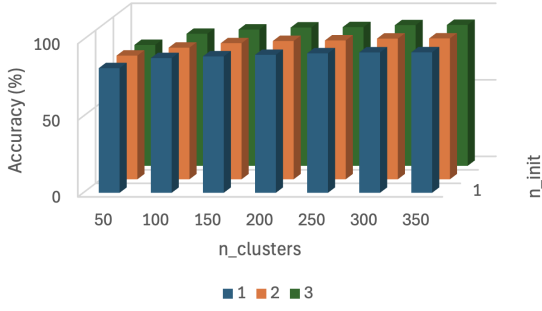
Table 4: Experiment 2 Results – Time and Accuracy across Parameter Settings

Seed	max_iter	n_init	n_clusters	TIME (s)	ACCURACY (%)
301	50	1	50	2.0	81.5
302	50	1	100	3.0	88.1
303	50	1	150	4.1	89.2
304	50	1	200	5.4	90.1
305	50	1	250	6.3	91.1
306	50	1	300	7.3	91.7
307	50	1	350	8.6	91.8
308	50	2	50	3.6	80.8
309	50	2	100	5.6	86.0
310	50	2	150	7.7	88.9
311	50	2	200	10.0	90.4
312	50	2	250	12.1	90.7
313	50	2	300	14.3	92.0
314	50	2	350	16.5	92.0
315	50	3	50	4.9	79.0
316	50	3	100	8.2	86.2
317	50	3	150	11.4	89.0
318	50	3	200	14.9	90.3
319	50	3	250	18.2	90.6
320	50	3	300	21.5	91.9

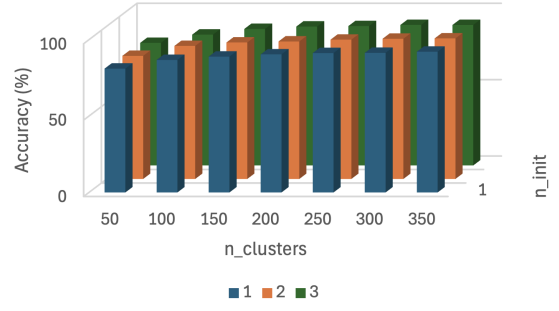
Table 4: Experiment 2 Results – continued

Seed	max_iter	n_init	n_clusters	TIME (s)	ACCURACY (%)
321	50	3	350	25.6	92.1
322	100	1	50	2.5	81.1
323	100	1	100	3.9	86.7
324	100	1	150	4.8	88.9
325	100	1	200	6.3	90.4
326	100	1	250	7.2	91.3
327	100	1	300	8.4	91.4
328	100	1	350	9.7	92.1
329	100	2	50	4.3	80.6
330	100	2	100	6.7	87.1
331	100	2	150	9.3	89.4
332	100	2	200	11.6	90.0
333	100	2	250	13.5	91.2
334	100	2	300	16.2	91.8
335	100	2	350	18.5	92.0
336	100	3	50	6.1	80.3
337	100	3	100	9.7	85.7
338	100	3	150	13.3	89.4
339	100	3	200	16.9	90.9
340	100	3	250	20.6	91.3
341	100	3	300	24.4	92.0
342	100	3	350	27.8	92.0

The results show that accuracy continued to improve as **n_clusters** increased, though with diminishing returns at higher values. With **max_iter**=50 and **n_init**=1, accuracy improved from 81.45% at 50 clusters to 91.80% at 350 clusters. For **max_iter**=100, accuracy reached a peak of 92.12% at 350 clusters. The largest gain was observed between 50 and 100 clusters, where accuracy increased by more than 6 percentage points; beyond 250 clusters, the improvements were mostly less than 1 percentage point per additional 50 clusters, indicating a plateau.



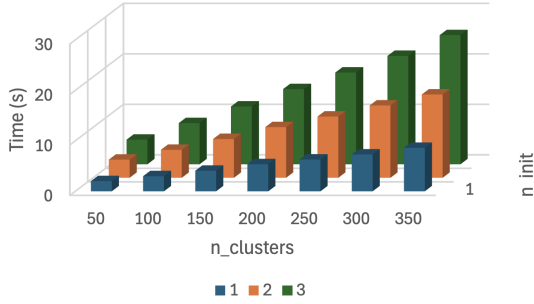
(a) graph of `n_cluster` and `n_init` against accuracy, with `max_iter` of 50



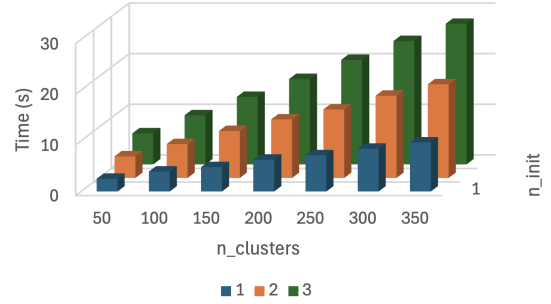
(b) graph of `n_cluster` and `n_init` against accuracy, with `max_iter` of 100

Figure 5: Processed accuracy graphs across parameter settings.

Runtime scaled almost linearly with both `n_clusters` and `n_init`. For `max_iter`=50 and `n_init`=1, runtime increased from 2.0 seconds at 50 clusters to 8.6 seconds at 350 clusters. With `n_init`=3, runtime rose more steeply from 4.9 seconds at 50 clusters to 25.6 seconds at 350 clusters. Doubling `n_init` approximately doubled runtime with only marginal accuracy improvement, confirming that additional initializations contributed little to clustering quality. Similarly, increasing `max_iter` from 50 to 100 led to runtimes that were slightly higher, but the accuracy gain remained under 0.5 percentage points, indicating that convergence was typically reached well before 100 iterations.



(a) graph of `n_cluster` and `n_init` against time, with `max_iter` of 50



(b) graph of `n_cluster` and `n_init` against time, with `max_iter` of 100

Figure 6: Processed time graphs across parameter settings.

Increasing `max_iter` from 50 to 100 produced negligible accuracy improvements, typically less than 0.5%. This confirmed the earlier finding from Experiment 1 that most clustering runs converge well before 100 iterations.

Beyond overall performance, digit-level accuracy revealed systematic variation. Digits such as “0” and “2” consistently achieved average accuracies above 95%, while digits such as “5” and especially “9” were more difficult, with averages of 83.7% and 76.7% respectively.

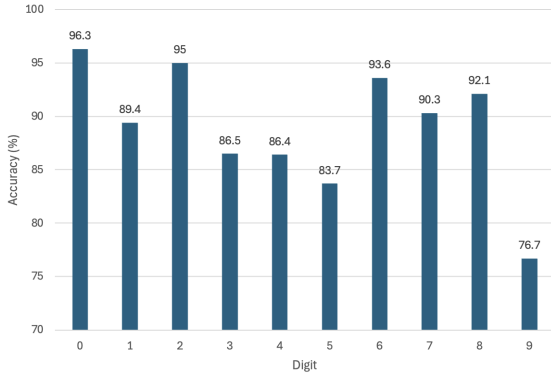
These discrepancies suggest that certain handwritten digit shapes are inherently harder to cluster with K-Means due to overlapping structures. At the same time, the maximum accuracy for each digit was consistently higher, showing that under favorable parameter settings, even the more challenging digits could be recognized with accuracies above 85–90%.

Table 5: Experiment 2 — Accuracy per Digit (%)

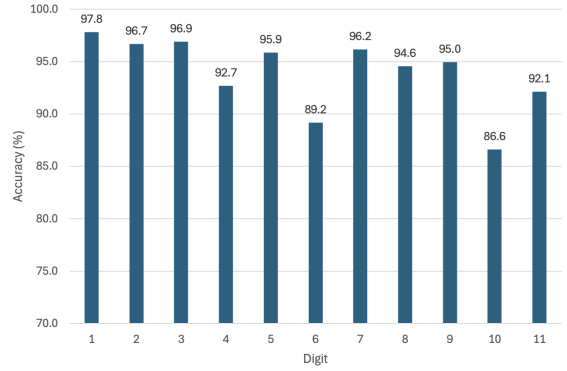
Seed	0	1	2	3	4	5	6	7	8	9	Total
301	91.7	92.3	95.8	78.4	58.3	79.8	88.9	84.4	87.0	58.6	81.4
302	96.1	93.1	95.7	86.4	88.7	76.3	94.6	90.7	89.9	72.3	88.0
303	96.3	87.0	96.2	92.7	85.9	82.8	94.2	92.2	88.9	77.0	89.1
304	96.3	89.3	95.9	85.4	84.2	87.0	95.8	92.5	94.1	82.1	90.1
305	96.6	95.7	95.7	89.5	89.5	82.8	90.2	93.0	94.0	84.0	91.1
306	97.2	91.9	96.3	89.0	93.3	86.6	93.9	93.7	94.7	82.2	91.7
307	97.2	92.2	96.0	89.0	91.3	87.1	94.6	94.3	92.7	83.8	91.8
308	94.7	76.3	95.5	78.5	70.6	73.6	91.0	90.0	89.3	57.9	80.8
309	94.5	84.1	94.7	86.4	91.0	75.2	94.3	90.2	93.4	66.8	86.0
310	96.9	95.1	89.7	87.1	84.4	83.1	94.0	91.1	93.5	75.2	88.9
311	96.5	88.6	96.0	88.5	91.4	84.7	92.8	92.8	93.6	80.4	90.4
312	97.1	91.3	96.2	89.0	87.9	84.9	93.1	93.1	93.4	81.8	90.7
313	97.3	92.3	95.7	89.4	87.8	88.0	95.8	93.8	94.6	85.1	91.9
314	96.3	92.4	95.6	91.0	92.2	88.1	95.5	94.6	94.3	81.5	92.0
315	94.9	75.0	95.7	83.1	71.5	77.4	91.6	65.7	89.0	51.4	79.0
316	96.2	84.4	94.8	83.8	87.2	76.5	92.9	87.6	91.7	71.7	86.2
317	96.4	86.8	96.1	85.1	90.3	81.9	94.7	93.5	90.5	77.8	89.0
318	97.0	95.9	91.1	82.9	88.8	84.9	95.9	92.5	92.1	82.3	90.3
319	97.1	96.0	91.3	86.6	84.9	86.0	95.3	92.3	92.8	82.9	90.6
320	96.5	92.1	95.2	91.4	89.2	86.1	96.2	93.4	94.1	84.5	91.9
321	96.0	92.3	96.9	90.2	92.2	89.2	94.3	94.3	93.3	82.8	92.1
322	94.3	92.3	95.7	79.2	75.5	75.0	76.9	79.1	87.7	58.2	81.1
323	95.7	84.4	95.9	88.3	87.0	73.3	94.7	89.6	87.8	75.0	86.7
324	96.4	87.6	95.9	84.4	88.2	82.9	91.1	91.4	93.3	79.5	88.9
325	97.0	89.5	94.2	87.3	87.4	87.3	95.1	93.9	93.6	79.7	90.4
326	96.5	92.1	96.1	88.3	90.7	87.5	95.3	93.6	92.2	80.8	91.2
327	97.2	91.4	96.5	88.8	90.9	88.0	94.6	87.6	93.9	85.5	91.4
328	96.9	92.9	94.6	90.8	90.7	88.8	94.8	91.8	92.9	86.6	92.1
329	95.5	75.9	95.2	82.8	68.4	77.4	92.3	73.9	88.0	58.5	80.6
330	95.5	83.8	95.5	79.2	82.9	87.1	94.4	92.5	88.7	74.9	87.1

Table 5: Experiment 2 — Accuracy per Digit (%) (continued)

Seed	0	1	2	3	4	5	6	7	8	9	Total
331	96.5	85.5	96.0	88.7	93.4	85.2	94.1	93.3	89.5	75.7	89.4
332	97.0	89.6	95.3	86.5	89.0	82.6	95.1	92.8	94.5	79.6	90.0
333	97.2	96.1	92.0	89.3	95.9	87.4	95.3	92.5	93.4	76.1	91.2
334	97.8	96.3	92.4	88.8	89.2	86.3	95.6	92.5	93.9	84.7	91.8
335	96.7	96.4	96.6	92.0	92.6	88.4	90.2	90.4	93.7	83.3	92.0
336	95.4	76.5	95.1	70.3	75.2	77.7	91.4	77.8	91.2	60.6	80.3
337	96.0	85.1	94.9	82.0	83.3	80.1	94.0	87.0	89.6	67.7	85.7
338	96.7	86.4	95.9	85.3	85.3	87.9	95.1	92.5	90.4	79.7	89.4
339	97.1	89.2	93.8	87.3	90.3	87.5	95.7	92.9	94.0	82.6	90.9
340	97.0	91.0	96.3	90.1	93.4	86.1	93.6	93.6	92.5	80.2	91.2
341	97.2	96.7	92.4	88.4	89.1	87.4	95.6	92.6	95.0	85.2	92.0
342	96.2	89.6	95.6	90.7	90.1	88.8	96.0	94.2	94.2	85.0	92.0
AVERAGE	96.3	89.4	95.0	86.5	86.4	83.7	93.6	90.3	92.1	76.7	



(a) Average Accuracy of each digit



(b) Maximum Accuracy of each digit

Figure 7: Accuracy of each digit achieved during Experiment 2

Taken together, these results confirm that `n_clusters` is the dominant parameter for improving clustering quality, while `n_init` and `max_iter` have comparatively minor effects. In practical terms, this means that accuracy gains can be maximized by increasing the number of clusters up to around 300, while avoiding unnecessary increases in `n_init` or `max_iter`, which only add to computation time. The balance between runtime and accuracy suggests that configurations such as `n_clusters=200–250`, `n_init=1`, and `max_iter=50` already deliver strong results in under 10 seconds, while higher cluster counts provide diminishing returns.

In summary, Experiment 2 demonstrated that careful refinement of the parameter space can deliver comparable or slightly improved accuracy relative to Experiment 1, while substantially reducing computational cost. Whereas Experiment 1 provided a broad exploration across wide parameter ranges, Experiment 2 showed that similar conclusions could be reached more efficiently by narrowing the search to the most impactful settings. This progression from broad exploration to targeted optimization offered a clearer

5 Conclusion

This investigation addressed the research question: *To what extent can the performance of the K-Means clustering algorithm be optimized on the MNIST dataset using different parameter settings in `scikit-learn`?* The results of the two-experiment design demonstrate that performance can indeed be optimized substantially through careful tuning of parameters, though improvements are bounded by diminishing returns and dataset-specific limitations.

Across both experiments, `n_clusters` emerged as the dominant parameter, with accuracy improving from roughly 57% at 10 clusters to a maximum of about 92% at 300–350 clusters. The largest gains were realized between 50 and 150 clusters, while improvements beyond 250 clusters were marginal, indicating an upper limit to the benefit of increasing cluster granularity. In contrast, reducing `n_init` from 10–50 (Experiment 1) to 1–3 (Experiment 2) showed that higher initialization counts offered little accuracy benefit but nearly tripled computation time. Similarly, increasing `max_iter` beyond 100 did not meaningfully improve clustering results, as most runs converged well before reaching this threshold. These findings establish that optimal configurations balance accuracy and efficiency by prioritizing higher cluster counts while minimizing unnecessary initializations and iteration caps.

Digit-wise analysis further clarified the extent of optimization. Certain digits such as ‘0’, ‘2’, and ‘6’ consistently achieved accuracies above 95%, even under modest parameter settings, while more ambiguous digits such as ‘5’ and ‘9’ remained difficult, averaging 83.7% and 76.7% respectively. Nevertheless, maximum digit accuracies exceeded 90% across all classes under favorable settings, indicating that clustering performance can be raised substantially but is unevenly distributed among digits.

Computational efficiency was also a critical dimension of optimization. Runtime scaled linearly with both `n_clusters` and `n_init`, ranging from as little as 2 seconds at 50 clusters and one initialization to nearly 28 seconds at 350 clusters and three initializations. A Pareto analysis showed that near-optimal configurations (e.g., 200–250 clusters with `n_init`=1 and `max_iter`=50) achieved over 90% accuracy in under 10 seconds, whereas pushing to 92% required runtimes two to three times longer. Thus, the degree of optimization depends on whether accuracy or efficiency is prioritized.

In conclusion, the performance of K-Means on MNIST can be optimized to achieve accuracies of around 92% through careful parameter selection, particularly by tuning the number of clusters. However, the extent of this optimization is constrained by diminishing accuracy gains, uneven digit performance, and linearly increasing runtime costs. While the findings are specific to the relatively clean MNIST dataset, the structured two-phase methodology developed here—broad exploration followed by targeted refinement—offers a generalizable framework for parameter tuning in unsupervised learning. Future work

should extend this approach to noisier datasets and alternative clustering algorithms to test robustness and scalability.

6 Evaluation

6.1 Evaluation of the Investigation

Overall, this investigation demonstrated that the performance of K-Means clustering on MNIST can be optimized substantially by tuning key parameters. The two-experiment design allowed both broad coverage and targeted refinement, ensuring that results reflected systematic patterns rather than isolated runs. The analysis confirmed that `n_clusters` was the dominant factor in accuracy, while `n_init` and `max_iter` contributed comparatively little beyond small values. Digit-wise performance revealed that optimization benefits were uneven, with some digits (e.g., '0', '2', '6') consistently exceeding 95% accuracy, while others (e.g., '5', '9') remained more challenging. This is mainly because the K-means algorithm expects clusters to be spherical, of similar variance, and well separated under Euclidean distance. However, the MNIST digit space does not always meet these conditions. Certain digits share overlapping structures, for example '5' and '9', which make their cluster imbalances or non-spherical. As a result, K-Means struggles to assign them consistently, even under optimized parameter settings. Runtime analysis further showed that optimization must balance accuracy with efficiency: while the highest accuracies of $\sim 92\%$ were obtained at 300–350 clusters, strong results above 90% were achievable in less than 10 seconds at lower cluster counts, demonstrating a clear trade-off between time and performance. These findings highlight both the extent and the limits of optimization within the assumptions of the K-Means algorithm and the properties of the MNIST dataset.

6.2 Strengths and Limitations

Strengths

- **Structured design:** The two-experiment approach progressed logically from broad exploration to refined testing, ensuring that parameter adjustments were evidence-based.
- **Comprehensive coverage:** Experiment 1 tested wide parameter ranges, while Experiment 2 concentrated on the most impactful areas, reducing redundancy without losing insight.
- **Efficiency gains:** Experiment 2 achieved equal or better accuracy compared to Experiment 1 with far lower computational cost, demonstrating effective optimization.
- **Clear link between findings and adjustments:** Observed diminishing returns in initialization and iteration limits directly informed the more efficient parameter choices in Experiment 2.

Limitations

- **Dataset simplicity:** MNIST is clean and balanced, meaning results may overstate K-Means performance relative to more complex, noisy, or real-world image data.
- **Single dataset:** Focusing exclusively on MNIST limits the generalizability of findings to datasets with different characteristics such as varied scales, handwriting styles, or background noise.
- **Algorithm constraints:** K-Means assumes spherical, equally sized clusters and relies on Euclidean distance, which may not align with the true structure of digit data. Other algorithms (e.g., DBSCAN, spectral clustering) may better capture nonlinear patterns.
- **Uneven digit performance:** Certain digits remained difficult to optimize, with accuracies lagging behind even under favorable settings, suggesting dataset-intrinsic challenges.

7 Works Cited

- Arvai, Kevin. “K-Means Clustering in Python: A Practical Guide.” *Real Python*, realpython.com/k-means-clustering-python/. Accessed 26 Aug. 2025.
- Deng, Li. “The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web].” *IEEE Signal Processing Magazine*, vol. 29, 2012, pp. 141–142.
- Ester, Martin, et al. “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise.” *Knowledge Discovery and Data Mining*, 1996.
- Fritzke, Bernd. “The K-means-u* Algorithm: Non-local Jumps and Greedy Retries Improve K-means++ Clustering.” *arXiv*, <https://doi.org/10.48550/ARXIV.1706.09059>.
- Hidalgo, José María Gómez, and Gonzalo Alvarez. “CAPTCHAs.” *Advances in Computers*, 2011, pp. 109–181. *ScienceDirect*, <https://doi.org/10.1016/b978-0-12-385510-7.00003-5>.
- Jain, Anil K. “Data Clustering: 50 Years beyond K-means.” *Pattern Recognition Letters*, vol. 31, no. 8, June 2010, pp. 651–666. *ScienceDirect*, <https://doi.org/10.1016/j.patrec.2009.09.011>.
- Jiang, Tammy, et al. “Supervised Machine Learning: A Brief Primer.” *Behavior Therapy*, vol. 51, no. 5, Sept. 2020, pp. 675–687. *National Library of Medicine*, <https://doi.org/10.1016/j.beth.2020.05.002>.
- “KMeans.” *scikit-learn*, scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html.
- LeCun, Y., et al. “Gradient-based Learning Applied to Document Recognition.” *Proceedings of the IEEE*, vol. 86, no. 11, 1998, pp. 2278–2324. *ResearchGate*, <https://doi.org/10.1109/5.726791>.
- LeCun, Yann, et al. “Deep Learning.” *Nature*, vol. 521, no. 7553, 27 May 2015, pp. 436–444. *ResearchGate*, <https://doi.org/10.1038/nature14539>.
- MacQueen, J. “Some Methods for Classification and Analysis of Multivariate Observations.” 1967.
- Mitchell, Tom M., and Eric M. Munson. *Machine Learning*. WCB/McGraw-Hill, 1997.

- Murtagh, Fionn, and Pedro Contreras. “Algorithms for Hierarchical Clustering: An Overview.” *WIREs Data Mining and Knowledge Discovery*, vol. 2, no. 1, 7 Dec. 2011, pp. 86–97. *ResearchGate*, <https://doi.org/10.1002/widm.53>.
- Noury, Zahra, and Mahdi Rezaei. “Deep-CAPTCHA: A Deep Learning Based CAPTCHA Solver for Vulnerability Assessment.” *arXiv*, 2020, arxiv.org/abs/2006.08296.
- Rousseeuw, Peter J. “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis.” *Journal of Computational and Applied Mathematics*, vol. 20, Nov. 1987, pp. 53–65. *ResearchGate*, [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).
- “2.3. Clustering.” *scikit-learn*, scikit-learn.org/stable/modules/clustering.html#k-means.
- Wani, Aasim Ayaz. “Comprehensive Analysis of Clustering Algorithms: Exploring Limitations and Innovative Solutions.” *PeerJ Computer Science*, vol. 10, 29 Aug. 2024, p. e2286. *PeerJ Computer Science*, <https://doi.org/10.7717/peerj-cs.2286>.
- Xiao, Han, et al. “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms.” *arXiv*, [abs/1708.07747](https://arxiv.org/abs/1708.07747) (2017).
- Xu, Rui, and Donald C. Wunsch II. “Survey of Clustering Algorithms.” *IEEE Transactions on Neural Networks*, vol. 16, no. 3, May 2005, pp. 645–678. *ResearchGate*, <https://doi.org/10.1109/tnn.2005.845141>.

8 Appendices

A. Code for Experiment 1

```
import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter
from sklearn.datasets import fetch_openml
from sklearn.cluster import KMeans
from sklearn.metrics import (
    pairwise_distances_argmin,
    adjusted_rand_score,
    adjusted_mutual_info_score,
    silhouette_score
)

np.random.seed(0)
seed = 1
n_clusters = 10 #10, 25, 50, 75, 100

save_path = f"figures/mnist/iter100/num{seed}"
os.makedirs(save_path, exist_ok=True)

# Load MNIST
X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
X_images = X.to_numpy().reshape(X.shape[0], 28, 28)
X = X_images.reshape(X.shape[0], -1)
y = y.astype(int).to_numpy()

kmeans = KMeans(max_iter= 100, #100, 300, 500
                 n_init= 10, #10, 20, 50
                 n_clusters=n_clusters,
                 algorithm='elkan',
                 random_state=seed)
```

```

# || TIMER STARTS
start_time = time.time()
cluster_labels = kmeans.fit_predict(X)
nearest_instances_idx =
    ↪ pairwise_distances_argmin(kmeans.cluster_centers_, X)
algorithm_time = time.time() - start_time
# || TIMER ENDS

n_clusters = len(np.unique(cluster_labels))

def display_digit_examples():
    rows = int(np.ceil(n_clusters / 10))
    fig, axes = plt.subplots(rows, 10, figsize=(10, rows * 1.5))
    axes = axes.flatten()

    for i in range(n_clusters):
        digit_idx = np.where(y == i)[0]
        if len(digit_idx) == 0: continue

        img = X_images[digit_idx[0]]
        if img.shape != (28, 28):
            img = img.reshape(28, 28)

        axes[i].imshow(img, cmap='gray')
        axes[i].set_xticks([])
        axes[i].set_yticks([])
        axes[i].set_title(str(i), fontsize=8)

    for j in range(i + 1, len(axes)):
        axes[j].axis('off')

    fig.tight_layout()
    fig.savefig(f"{save_path}/digits_improved_layout.png")

def generate_bar_charts():
    cluster_info = []
    for i in range(n_clusters):
        indices = np.where(cluster_labels == i)[0]
        label_counts = Counter(y[indices])

```

```

        clean_counts = {int(k): v for k, v in label_counts.items()}
        nearest_label = int(y[nearest_instances_idx[i]])
        cluster_info.append((nearest_label, i, clean_counts))

cluster_info.sort()

fig, axes = plt.subplots(n_clusters, 1, figsize=(8, n_clusters * 2.5))
for idx, (nearest_label, cluster_id, label_distribution) in
    ↪ enumerate(cluster_info):
    labels = sorted(label_distribution.keys())
    counts = [label_distribution[k] for k in labels]
    axes[idx].bar(labels, counts)
    axes[idx].set_title(f"Cluster {cluster_id} (Nearest Label:
        ↪ {nearest_label})")
    axes[idx].set_xlabel("Digit Label")
    axes[idx].set_ylabel("Count")
    axes[idx].set_xticks(range(10))
fig.tight_layout()
fig.savefig(f"{save_path}/cluster_label_distribution_by_nearest.png",
    ↪ bbox_inches='tight')

def display_scores():
    ami = adjusted_mutual_info_score(cluster_labels, y)
    ari = adjusted_rand_score(cluster_labels, y)
    sil = silhouette_score(X, cluster_labels)
    print(f"AMI: {ami:.4f}, ARI: {ari:.4f}, Silhouette: {sil:.4f}, Time:
        ↪ {algorithm_time:.2f}s")
    with open(f"{save_path}/metrics_and_params.txt", "w") as f:
        f.write(f"AMI: {ami}\n")
        f.write(f"ARI: {ari}\n")
        f.write(f"Silhouette: {sil}\n")
        f.write(f"Time (s): {algorithm_time:.2f}\n")
        f.write(f"Params: n_clusters={n_clusters}, n_init={50},
            ↪ max_iter=300, "
                f"algorithm='elkan', seed={seed}\n")

def display_cluster_centers():
    rows = int(np.ceil(n_clusters / 10))
    fig, axes = plt.subplots(rows, 10, figsize=(10, rows * 1.5))
    axes = axes.flatten()

```

```

for i in range(n_clusters):
    axes[i].imshow(kmeans.cluster_centers_[i].reshape(28, 28),
        ↪ cmap='gray')
    axes[i].set_xticks([]); axes[i].set_yticks([])
    axes[i].set_title(str(i), fontsize=8)
for j in range(i + 1, len(axes)):
    axes[j].axis('off')
fig.tight_layout()
fig.savefig(f"{save_path}/cluster_centers.png")

def display_nearest_instances():
    rows = int(np.ceil(n_clusters / 10))
    fig, axes = plt.subplots(rows, 10, figsize=(10, rows * 1.5))
    axes = axes.flatten()
    for i in range(n_clusters):
        axes[i].imshow(X_images[nearest_instances_idx[i]], cmap='gray')
        axes[i].set_title(str(y[nearest_instances_idx[i]]), fontsize=8)
        axes[i].set_xticks([]); axes[i].set_yticks([])
    for j in range(i + 1, len(axes)):
        axes[j].axis('off')
    fig.tight_layout()
    fig.savefig(f"{save_path}/nearest_instances.png")

def display_cluster_infos():
    matrix = np.zeros((10, 10))
    cluster_info = []
    nearest_label_to_counts = {}

    for i in range(n_clusters):
        idxs = np.where(cluster_labels == i)[0]
        counts = Counter(y[idxs])
        total = sum(counts.values())

        for k, v in counts.items():
            matrix[y[nearest_instances_idx[i]]][k] += v

    nearest_label = y[nearest_instances_idx[i]]
    summary = ", ".join(f"{k}: {v} ({v/total:.1%})" for k, v in
        ↪ sorted(counts.items(), key=lambda x: -x[1]))

```

```

cluster_info.append((i, nearest_label, summary))
if nearest_label not in nearest_label_to_counts:
    nearest_label_to_counts[nearest_label] = Counter()
nearest_label_to_counts[nearest_label].update(counts)

df = pd.DataFrame(cluster_info, columns=["Cluster", "Nearest Label",
    ↪ "Label Distribution Summary"])
df.to_csv(f"{save_path}/cluster_info.csv", index=False)

grouped_info = []
for nearest_label, counter in nearest_label_to_counts.items():
    total = sum(counter.values())
    summary = ", ".join(f"{k}: {v} ({v/total:.1%})" for k, v in
    ↪ sorted(counter.items(), key=lambda x: -x[1]))
    grouped_info.append((nearest_label, summary))

grouped_df = pd.DataFrame(grouped_info, columns=["Nearest Label",
    ↪ "Combined Label Distribution Summary"])
grouped_df.to_csv(f"{save_path}/nearest_label_grouped_summary.csv",
    ↪ index=False)

np.savetxt(f"{save_path}/matrix_output.txt", matrix, fmt="%d",
    ↪ delimiter='\t')

# Run
display_scores()
#display_digit_examples()
display_cluster_centers()
display_nearest_instances()
display_cluster_infos()
#generate_bar_charts()

```

B. Code for Experiment 2

```

import os
import time
import numpy as np
import pandas as pd

```

```

import matplotlib.pyplot as plt
from collections import Counter
from sklearn.datasets import fetch_openml
from sklearn.cluster import KMeans
from sklearn.metrics import (
    pairwise_distances_argmin,
    adjusted_rand_score,
    adjusted_mutual_info_score,
    silhouette_score
)

X, y = fetch_openml('mnist_784', version=1, return_X_y=True)
X_images = X.to_numpy().reshape(X.shape[0], 28, 28)
X = X_images.reshape(X.shape[0], -1)
y = y.astype(int).to_numpy()

def run_experiment(seed, n_clusters, n_init, max_iter):
    save_path = f"figures/mnist/newRange/seed{seed}"
    os.makedirs(save_path, exist_ok=True)

    kmeans = KMeans(
        max_iter=max_iter,
        n_init=n_init,
        n_clusters=n_clusters,
        algorithm='elkan',
        random_state=seed
    )

    start_time = time.time()
    cluster_labels = kmeans.fit_predict(X)
    nearest_instances_idx =
        ↪ pairwise_distances_argmin(kmeans.cluster_centers_, X)
    algorithm_time = time.time() - start_time

    ami = adjusted_mutual_info_score(cluster_labels, y)
    ari = adjusted_rand_score(cluster_labels, y)
    sil = silhouette_score(X, cluster_labels)

    with open(f"{save_path}/metrics_and_params.txt", "w") as f:

```

```

f.write(f"AMI: {ami}\n")
f.write(f"ARI: {ari}\n")
f.write(f"Silhouette: {sil}\n")
f.write(f"Time (s): {algorithm_time:.2f}\n")
f.write(f"Params: n_clusters={n_clusters}, n_init={n_init},
↪ max_iter={max_iter}, seed={seed}\n")

print(f"AMI: {ami:.4f}, ARI: {ari:.4f}, Silhouette: {sil:.4f}, Time:
↪ {algorithm_time:.2f}s")

rows = int(np.ceil(n_clusters / 10))
fig, axes = plt.subplots(rows, 10, figsize=(10, rows * 1.5))
axes = axes.flatten()
for i in range(n_clusters):
    axes[i].imshow(kmeans.cluster_centers_[i].reshape(28, 28),
↪ cmap='gray')
    axes[i].set_xticks([]); axes[i].set_yticks([])
    axes[i].set_title(str(i), fontsize=8)
for j in range(i + 1, len(axes)):
    axes[j].axis('off')
fig.tight_layout()
fig.savefig(f"{save_path}/cluster_centers.png")
plt.close(fig)

fig, axes = plt.subplots(rows, 10, figsize=(10, rows * 1.5))
axes = axes.flatten()
for i in range(n_clusters):
    axes[i].imshow(X_images[nearest_instances_idx[i]], cmap='gray')
    axes[i].set_title(str(y[nearest_instances_idx[i]]), fontsize=8)
    axes[i].set_xticks([]); axes[i].set_yticks([])
for j in range(i + 1, len(axes)):
    axes[j].axis('off')
fig.tight_layout()
fig.savefig(f"{save_path}/nearest_instances.png")
plt.close(fig)

matrix = np.zeros((10, 10))
for i in range(n_clusters):
    idxs = np.where(cluster_labels == i)[0]

```

```

        counts = Counter(y[idxs])
        for k, v in counts.items():
            matrix[y[nearest_instances_idx[i]]][k] += v
    np.savetxt(f"{save_path}/matrix_output.txt", matrix, fmt="%d",
        ↪ delimiter='\t')

#LOOP
seed = 301
n_clusters_list = [50, 100, 150, 200, 250, 300, 350]
#n_clusters_list = [10, 25, 50, 75, 100, 125, 150, 200]
n_init_list = [1, 2, 3]
#max_iter_list = [100]
max_iter_list = [50, 100]

for max_iter in max_iter_list:
    for n_init in n_init_list:
        for n_clusters in n_clusters_list:
            #print(f"Running: seed={seed}, clusters={n_clusters},
            ↪ n_init={n_init}, max_iter={max_iter}")
            run_experiment(seed, n_clusters, n_init, max_iter)
            seed +=1

```

C. Code for generating accuracy

```

#include <iostream>
#include <vector>
#include <fstream>
using namespace std;

bool call(int a){
    string s = to_string(a);

    std::ifstream infile(("../../Desktop/IB/CS
    ↪ EE/Code3.0/figures/mnist/newRange/seed"+s+"/matrix_output.txt"));
    std::ofstream outfile(("../../Desktop/IB/CS
    ↪ EE/Code3.0/figures/mnist/newRange/seed"+s+"/accuracy.txt"));

```

```

if (!infile) {
    cerr << "Error: Could not open input file\n";
    return false;
}
if (!outfile) {
    cerr << "Error: Could not open output file\n";
    return false;
}

int rows = 10;
int cols = 10;
vector<vector<int>> data(rows, vector<int>(cols));
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        infile >> data[i][j];
    }
}

int ok = 0, total = 0;
for (int i = 0; i < rows; ++i) {
    int yes = 0, no = 0;
    for (int j = 0; j < cols; ++j) {
        if (i == j) yes += data[i][j];
        else no += data[i][j];
    }
    double acc = (double(yes) * 100) / double(yes + no);
    outfile << (i + 1) << ": " << acc << "%\n";

    ok += yes;
    total += yes + no;
}

double total_acc = (double(ok) * 100) / double(total);
cout << total_acc << "%\n";
outfile << "total accuracy: " << total_acc << "%\n";
return true;
}

```

```
int main() {  
    for (int i=301; i<=342; i++){  
        if (!call(i)){  
            cout<<"ERROR at: "<<i<<endl;  
            break;  
        }  
    }  
}
```